

Manual de Contenidos — Parcial 2: SNA y Sistemas Recomendadores

Fernando Luis Pinilla Molina

Cómo leer este manual

- **[CAE EN EXAMEN]** marca patrones que se repiten en TODOS los ejercicios de clase (Airports, Reddit, SNA-Tools, sentimientos, recomendadores). Es lo que tienes que dominar a ciegas.
 - **[PROYECTO AMAZON]** marca lo que aplicarás al ABP.
 - son trampas o detalles que se piden corregir / vienen como “warning” en igraph 2.x.
 - El resto es teoría de apoyo. Mismo estilo que el manual del tema pasado.
-

Librerías canónicas del bloque

```
# STACK DE SNA -----
library(igraph)      # MOTOR (núcleo en C, rápido)
library(tidygraph)   # PUENTE (red como dos tibbles nodes/edges + verbos dplyr)
library(ggraph)      # ARTISTA (gramática ggplot para grafos)
library(igraphdata)  # Datasets clásicos (karate, USairports, ...)

# DATA WRANGLING -----
library(tidyverse)   # dplyr, ggplot2, purrr, tibble, ...
library(data.table)  # fread() para datasets grandes ( data.frame)
library(Matrix)      # Matrices dispersas (biadyacencia)
library(readxl)      # read_excel() para xlsx (Tronos)

# NLP / REDES SEMÁNTICAS -----
library(tidytext)    # unnest_tokens(), stop_words, get_stopwords()
library(SnowballC)   # wordStem() (stemming en español)
```

```
library(widyr)      # pairwise_cor() para co-ocurrencias
library(ggforce)    # geom_mark_hull() para envolturas convexas

# RECOMENDADORES Y NUMÉRICA -----
library(knitr)      # kable() para tablas en informes
library(scales)     # label_dollar(), label_percent() en ggplot
```

Mantra del stack SNA: `igraph` calcula, `tidygraph` ordena, `ggraph` pinta. Cada uno con su responsabilidad.

PARTE I — Análisis de Redes Sociales (SNA)

1. Representación de Grafos

Las dos formas equivalentes

A) Matriz de adyacencia ($A_{n \times n}$) — relación con álgebra lineal:

$$(A^k)_{ij} = \text{nº de caminos de longitud exactamente } k \text{ entre } i \text{ y } j$$

```
A <- matrix(c(0,1,0,
              1,0,1,
              1,0,0), nrow = 3, byrow = TRUE)
dimnames(A) <- list(c("A","B","C"), c("A","B","C"))
A2 <- A %*% A      # caminos de longitud 2
```

B) Edge list — la forma natural en proyectos reales: una tabla con columnas `from`, `to` (+ atributos `weight`, `distancia`, ...).

[CAE EN EXAMEN] — Para construir un grafo desde `data.frames`, **renombra siempre** las dos columnas de origen/destino a `from` y `to` y ponlas como las dos PRIMERAS columnas. Si no, `graph_from_data_frame()` falla en silencio.

```
# Patrón que aparece en Airports, Reddit y Tronos:
nodos <- data.frame(id = ..., ciudad = ..., pais = ..., pasajeros_m = ...)
aristas <- data.frame(from = ..., to = ..., distancia = ...) # ¡PRIMERAS COLUMNAS!

g <- graph_from_data_frame(d = aristas, vertices = nodos, directed = TRUE)
```

2. Construcción de Grafos en igraph

Método	Para qué
<code>graph_from_data_frame()</code>	El más usado en clase. Desde dos <code>data.frames</code> .
<code>graph_from_adjacency_matrix(A, mode="undirected")</code>	Desde una matriz binaria.
<code>make_graph(c("A", "B", "B", "C"))</code>	Por lista lineal de aristas (<code>from1,to1,from2,to2,...</code>).
<code>make_graph(~ A-B-C-A)</code>	Por fórmula (rápido para ejemplos a mano).
<code>make_graph("Zachary")</code>	Grafos famosos (Zachary = club de karate).
<code>sample_gnp(n, p)</code>	Erdős–Rényi (red aleatoria, control: probabilidad).
<code>sample_pa(n, power, m)</code>	Barabási–Albert (libre de escala, hubs).
<code>play_islands(n_islands, n, p_in, p_between)</code>	Comunidades sintéticas (útil para probar Louvain).

Inspección anatómica del grafo

[CAE EN EXAMEN] — Bloque que se repite literalmente en Airports y Reddit:

```
V(g)           # Vertex Sequence
E(g)           # Edge Sequence
vcount(g); ecount(g)  # n° nodos y aristas
is_directed(g)  # TRUE/FALSE
is_bipartite(g) # TRUE/FALSE

# Atributos
vertex_attr_names(g)  # lista de atributos de nodos
edge_attr_names(g)    # lista de atributos de aristas
V(g)$name             # acceso al atributo "name" de los nodos
E(g)$weight           # acceso al "weight" de las aristas

# Inspección de una arista concreta (Reto 2 de Reddit)
edge_attr(g, index = 15)  # todos los atributos de la arista 15
tail_of(g, E(g)[15])      # nodo origen de la arista 15
head_of(g, E(g)[15])      # nodo destino de la arista 15
```

Modificación de atributos (condicional)

[CAE EN EXAMEN] — Los tres patrones de modificación que aparecen en TODOS los ejercicios:

```
# (1) Asignación global de un valor
V(g)$color <- "lightblue"
V(g)$size <- V(g)$pasajeros_m / 4          # proporcional a un atributo

# (2) Asignación condicional con indexación lógica
indices_condicion <- E(g)$distancia > 2000
E(g)[indices_condicion]$color <- "red"
E(g)[!indices_condicion]$color <- "grey80"

# (3) MÉTODO PREFERIDO: ifelse() vectorizado (más limpio)
E(g)$color <- ifelse(E(g)$distancia > 2000, "red", "grey80")
V(g)$label <- V(g)$ciudad                  # cambiar las etiquetas
```

Operadores especiales de selección de aristas

Operador	Significado
A %--% B	Aristas entre A y B en cualquier dirección (no dirigido).
A %->% B	Aristas que NACEN en A y MUEREN en B (dirigido).
A %<-% B	Aristas que nacen en B y mueren en A.

```
isVowel <- V(g)$name %in% c("A","E","I","O","U")
v1 <- V(g)[isVowel]
E(g)[v1 %--% v1]$width <- 4          # engrosar aristas entre vocales
```

3. Filtrado del Grafo (poda)

[CAE EN EXAMEN] — Reto 2 de Airports y limpieza del proyecto Amazon:

```

# (1) Subgrafo por condición sobre nodos (lo más usado)
grados <- degree(g)
g_hubs <- induced_subgraph(g, vids = which(grados > 3))

# (2) Resta directa de nodos (más expresiva, dejar el negativo)
g_hubs2 <- g - V(g)[grados <= 3]

# (3) Eliminar aristas por peso (proyecto Amazon: umbral N)
g_pruned <- delete_edges(g, E(g)[weight < N])

# (4) Eliminar nodos aislados que quedan tras podar aristas
g_clean <- delete_vertices(g_pruned, V(g_pruned)[degree(g_pruned) == 0])

# (5) Simplificar: quitar bucles y multi-aristas
g <- simplify(g)

# (6) K-core: extraer el "esqueleto denso" (útil con >5000 nodos)
g_kcore <- induced_subgraph(g, V(g)[coreness(g) >= 5])

```

[PROYECTO AMAZON] — Pipeline de limpieza recomendado: `bipartite_projection`
→ `delete_edges(weight < N)` → `delete_vertices(degree == 0)` → (si >5000 nodos)
`induced_subgraph(coreness k)`.

4. Visualización con `plot()` (igraph clásico)

Layouts

```

plot(g, layout = layout_with_fr(g)) # Fruchterman-Reingold (estándar para comunidades)
plot(g, layout = layout_with_kk(g)) # Kamada-Kawai (cuando importan las distancias)
plot(g, layout = layout_in_circle(g)) # Circular (redes densas / cíclicas)
plot(g, layout = layout_on_grid(g)) # Cuadrícula (inspección individual)
plot(g, layout = layout_as_star(g)) # Estrella (para ego-networks)
plot(g, layout = layout_with_drl(g)) # DrL: escalable a miles de nodos

# Layout fijo para reutilizar:
coords <- layout_in_circle(g)
plot(g, layout = coords)

```

Dos plots lado a lado (Reto 3 de Airports)

```
par(mfrow = c(1, 2))          # 1 fila, 2 columnas
plot(g_hubs, layout = layout_in_circle(g_hubs), main = "Circular")
plot(g_hubs, layout = layout_with_fr(g_hubs), main = "FR")
par(mfrow = c(1, 1))          # resetear
```

Parámetros estéticos clave

```
plot(g,
  vertex.size      = 12,
  vertex.color     = V(g)$color,
  vertex.label     = V(g)$ciudad,
  vertex.label.cex = 0.7,
  vertex.label.dist = 1.2,          # separar etiqueta del círculo
  edge.color       = E(g)$color,
  edge.width       = E(g)$weight * 0.5,
  edge.curved      = 0.2,          # curva las aristas
  edge.arrow.size  = 0.4,          # tamaño flecha (dirigidos)
  mark.groups      = lista_de_grupos, # envoltura convexa por comunidad
  mark.col         = colores_transparentes,
  main             = "Título")
```

Aviso del enunciado de Amazon: con >5000 nodos los layouts force-directed se vuelven inviiables. Filtra primero, dibuja después.

5. El paradigma tidygraph

El método moderno y el que el profesor prefiere en los ejercicios. Convierte el grafo en dos tibbles vinculados (nodes, edges) y los manipulas con verbos dplyr.

Pipeline canónico

[CAE EN EXAMEN] — Aparece literalmente en Reddit Reto 4 y en clase 1 (Tronos / Capítulo 2):

```
library(tidygraph)
library(ggraph)

g_tidy <- as_tbl_graph(g)

g_tidy %>%
  activate(nodes) %>%                                # capa activa = nodos
  mutate(
    grado      = centrality_degree(),
    betweenness = centrality_betweenness(),
    pagerank    = centrality_pagerank(),
    closeness   = centrality_closeness(),
    eigen       = centrality_eigen(),
    comunidad   = as.factor(group_louvain())
  ) %>%
  filter(!node_is_isolated()) %>%                     # quitar aislados
  activate(edges) %>%                                  # capa activa = aristas
  mutate(puente = centrality_edge_betweenness()) %>%
  filter(puente > 2)                                    # podar aristas poco importantes
```

Sacar un líder (Reto 5 de Reddit)

[CAE EN EXAMEN] — Patrón estándar para responder “¿quién es el líder por X?”:

```
g_tidy %>%
  activate(nodes) %>%
  as_tibble() %>%                                     # convertir a tabla normal
  arrange(desc(pagerank)) %>%                         # ordenar por la métrica
  slice(1)                                             # o head(1)
```

`degree()` vs `centrality_degree()`

Función	Cuándo
<code>igraph::degree(g)</code>	Función “huérfana”. Le pasas <code>g</code> . El vector resultante NO se sincroniza si después filtras nodos.
<code>centrality_degree()</code> y demás <code>centrality_*()</code> / <code>group_*()</code> / <code>node_is_*()</code>	Solo dentro de <code>mutate()</code> / <code>filter()</code> sobre un <code>tbl_graph</code> . Sincronizadas con la tabla activa.

El pronombre mágico `.N()` (cruzar aristas con atributos de nodos)

```
karate_tidy %>%
  activate(edges) %>%
  mutate(
    fac_origen = .N()$Faction[from],    # atributo del nodo de ORIGEN
    fac_destino = .N()$Faction[to],      # atributo del nodo de DESTINO
    es_cruce_bando = fac_origen != fac_destino
  )
```

Morphing — vistas temporales

Morpher	Uso
<code>to_shortest_path(from, to)</code>	Camino más corto entre dos nodos
<code>to_local_neighborhood(node, order)</code>	Ego-network (saltos = <code>order</code>)
<code>to_contracted(atributo)</code>	Colapsa nodos del mismo grupo en súper-nodos
<code>to_components()</code>	Aísla componentes conexas
<code>to_minimum_spanning_tree()</code>	Esqueleto mínimo de conectividad
<code>to_linegraph()</code>	Convierte aristas en nodos

```
karate_tidy %>%
  morph(to_shortest_path, from = 1, to = 34) %>%
  activate(edges) %>%
  mutate(en_ruta_critica = TRUE) %>%
  unmorph()                                # consolida la marca en el grafo original
```


6. Métricas de Centralidad

Concepto clave: “importancia” no significa lo mismo en todos los contextos. La elección de métrica encarna una hipótesis sociológica.

[CAE EN EXAMEN] — La tabla siguiente es de aprendizaje obligatorio:

Métrica	igraph	tidygraph	Qué mide	Cuándo elegirla
Degree	degree(g) / degree(g, mode="in") / mode="out"	centrality_degree(g)	Nº de conexiones directas	Popularidad, hubs estructurales
Betweenness	betweenness(g)	centrality_betweenness(g)	Cuántos caminos mínimos pasan por el nodo	Puentes entre comunidades (SPOF)
Closeness	closeness(g)	centrality_closeness(g)	Inversa de la suma de distancias	Eficiencia de difusión
Harmonic	harmonic_centrality(g)		Suma de inversas	Sustituto correcto de closeness en grafos dirigidos/dis- connected
Eigenvector	eigen_centrality(g)	centrality_eigen(g)	Conectado a nodos importantes	Prestigio relativo
PageRank	page_rank(g)\$vector	centrality_pagerank(g)	Autovector dominante de la matriz de Google	Prestigio en dirigidos, “productos faro”
HITS Hub/Author- ity	hits_scores(g)\$hub, \$authority		Dual: hubs apuntan a authorities	Web, citas (dirigidos)

Heurística clave del analista

Elige **PageRank** / **Eigenvector** cuando el capital social sea **sumativo** (te lo conceden otros con prestigio). Elige **Betweenness** cuando el capital resida en la **posición estructural** (gatekeeper, broker).

Trampa clásica con `closeness()` en grafos dirigidos

Un nodo con UN ÚNICO arco saliente puede dar `closeness` = 1.0 (engañoso, porque “premia” a nodos marginales). **Siempre usa `harmonic_centrality()` en grafos dirigidos.**

[PROYECTO AMAZON] — El enunciado pide explícitamente:

- **PageRank** → “productos faro”.
 - **Betweenness** → “productos puente entre categorías”.
 - **Cruce con PCA** del Bloque I: ¿los outliers del biplot son los hubs del grafo?
-

7. Métricas Globales (Topometría)

[CAE EN EXAMEN] — Reto 3 de Reddit pide LITERALMENTE esto:

```
edge_density(g)          # aristas reales / aristas posibles [0, 1]
diameter(g)              # camino más corto MÁS LARGO de la red
mean_distance(g)         # distancia media entre todos los pares
transitivity(g, type = "global") # coef. clustering global
transitivity(g, type = "local")  # coef. clustering por nodo
count_components(g)        # nº de componentes conectados (¿hay islas?)
components(g)$csize       # tamaño de cada componente
components(g)$membership  # qué nodo pertenece a qué componente
```

Transitividad / Coeficiente de Clustering local

$$C_v = \frac{\text{triángulos centrados en } v}{\text{triples (V) centrados en } v}$$

Interpretación rápida:

- $C_v = 1$ sub-comunidad perfectamente cohesiva (clique).
- $C_v \approx 0$ el nodo es el único punto de contacto entre grupos = **broker** / **structural hole**.

[PROYECTO AMAZON] — La **densidad** es la métrica que el enunciado te pide para **justificar la elección de N** (umbral mínimo de usuarios compartidos): barre $N = 2, 3, 5, 10, \dots$ midiendo `edge_density()` y nº de nodos no aislados. Elige N que dé densidad manejable (típicamente 10^{-3} a 10^{-1}).

Otras métricas globales útiles

```
articulation_points(g)      # nodos cuya eliminación desconecta el grafo
cliques(g, min = 3)         # subconjuntos "todos con todos"
largest_cliques(g)          # cliques maximales
coreness(g)                 # k-core de cada nodo (poda agresiva)
```

8. Detección de Comunidades

Idea unificadora: una comunidad es un conjunto densamente conectado por dentro y débilmente con el exterior. Todos los algoritmos optimizan una variante de la **modularidad** $Q \in [-1, 1]$. Valores típicos buenos: $Q \in [0.3, 0.7]$.

Algoritmo	Función igraph	tidygraph	Notas
Louvain	cluster_louvain(g)	group_louvain()	Estocástico → <code>set.seed()</code> . Maximiza modularidad por agregación voraz. Rápido. Solo no dirigido.
Walktrap	cluster_walktrap(g, steps = 4)	group_walktrap()	Paseos aleatorios cortos: nodos que se “ven” en pocos pasos forman comunidad.
Infomap	cluster_infomap(g)	group_infomap()	Teoría de la información. Acepta pesos. Profesor lo usa en clase.
Edge Betweenness (Girvan-Newman)	cluster_edge_betweenness(g)	group_edge_betweenness()	Elimina iterativamente aristas con mayor betweenness. Lento pero pedagógico.

Algoritmo	Función igraph	tidygraph	Notas
Fast-Greedy	<code>cluster_fast_greedy(g)</code>	<code>group_fast_greedy()</code>	Greedy de modularidad. Rápido en redes grandes no dirigidas y sin pesos negativos.

Salida y manejo de un objeto `communities`

```
set.seed(2026)
comm <- cluster_louvain(g)

length(comm)      # n° de comunidades
sizes(comm)       # tamaño de cada comunidad
membership(comm)  # vector nodo -> comunidad
modularity(comm)  # calidad del particionamiento [0, ~1]
communities(comm) # lista de vectores por comunidad

plot(comm, g, main = "Comunidades (Louvain)")
```

[PROYECTO AMAZON] — Enunciado pide **Louvain o Walktrap** + análisis de sensibilidad:

- Para Louvain (igraph 2.0): varía **resolution** (0.5, 1.0, 1.5, 2.0). Baja → menos comunidades grandes; alta → más comunidades pequeñas.
- Para Walktrap: varía **steps** (2, 4, 6, 8).
- Reporta cómo cambian `modularity()` y `length(comm)` para cada valor.

9. Análisis de Texto y Redes Semánticas (pipeline NLP)

Aparece literalmente en el ejercicio de “sentimientos sobre la Ley de Vivienda”. El pipeline es **universal** para cualquier corpus.

Pipeline completo (lo que se evalúa)

```

library(tidytext); library(SnowballC); library(widyr); library(ggforce)

# 1. TOKENIZACIÓN
tokens <- textos |>
  unnest_tokens(word, texto)          # explota cada texto en filas (una palabra por fila)

# 2. LIMPIEZA: stopwords + stemming
stop_words_es <- get_stopwords(language = "es")
tokens_limpios <- tokens |>
  anti_join(stop_words_es, by = "word") |>
  filter(nchar(word) > 3) |>          # palabras muy cortas fuera
  mutate(stem = wordStem(word, language = "spanish"))

# 3. ANÁLISIS DE SENTIMIENTO (con lexicón de pares palabra→valor)
resultado <- tokens |>
  inner_join(lexicon_esp, by = "word") |>
  group_by(id) |>
  summarise(sentimiento = sum(valor))

# 4. CO-OCURRENCIA (red semántica)
word_cors <- tokens_limpios |>
  pairwise_cor(word, id, sort = TRUE) # correlación entre palabras por documento

# 5. GRAFO SEMÁNTICO + COMUNIDADES (Louvain)
red <- word_cors |>
  filter(correlation > 0) |>
  as_tbl_graph(directed = FALSE) |>
  mutate(comunidad = group_louvain())

# 6. VISUALIZACIÓN profesional con ggraph
ggraph(red, layout = "fr") +
  geom_mark_hull(aes(x = x, y = y, fill = comunidad, group = comunidad),
    alpha = 0.1, color = NA, expand = unit(3, "mm")) +
  geom_edge_link(aes(edge_alpha = correlation), edge_colour = "gray80") +
  geom_node_point(aes(color = comunidad), size = 5) +
  geom_node_text(aes(label = name), repel = TRUE, size = 3.5, fontface = "bold") +
  theme_graph() + theme(legend.position = "none")

```

Visualización del sentimiento (ggplot)

```
resultado |>
  ggplot(aes(x = id, y = sentimiento, fill = sentimiento > 0)) +
  geom_col() +
  geom_text(aes(label = sentimiento))
```

10. Ego Networks

Subgrafo formado por un nodo focal (“Ego”) y sus vecinos hasta cierto orden:

```
ego_red <- make_ego_graph(g, order = 1, nodes = 3)[[1]] # orden 1: directos
plot(ego_red, layout = layout_as_star)

make_ego_graph(g, order = 2, nodes = 3)[[1]] # orden 2: amigos de amigos
```

Aplicaciones: epidemiología (paciente cero), marketing de influencia, capital social.

PARTE II — Sistemas de Recomendación

11. Filtrado Colaborativo User-Based

Mantra: “Dime con quién andas y te diré qué te va a gustar”. Si dos usuarios coinciden en gustos pasados, podemos inferir gustos futuros.

Matriz de utilidad

R_{ui} = valoración del usuario u por el ítem i

Es **extremadamente dispersa** (>95–99% NA en producción).

[CAE EN EXAMEN] — Calcular **dispersión** (sparsity):

```
dispersion <- sum(is.na(M)) / (nrow(M) * ncol(M))
```

Similitud de Pearson sobre ítems comunes (cálculo a mano)

[CAE EN EXAMEN] — Esto se calcula a mano en el examen (PP1):

$$\text{Sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}}$$

Donde I_{uv} son los ítems valorados por **ambos**. La media $\bar{r}_u$ se calcula usando **solo los ítems en común** cuando el ejercicio te lo pide a mano (PP1 lo aclara así).

```
# Vectorial: cor() con manejo de NAs
r1 <- M["Usuario_1", ]; r2 <- M["Usuario_2", ]
sim_12 <- cor(r1, r2, use = "pairwise.complete.obs")
```

Matriz completa de similitudes

[CAE EN EXAMEN] — Función `similitud_usuarios` (memorízala):

```
similitud_usuarios <- function(M) {
  # cor() opera entre COLUMNAS, por eso transponemos para obtener sim entre USUARIOS
  sim <- suppressWarnings(cor(t(M), use = "pairwise.complete.obs"))
  sim[is.na(sim)] <- 0 # NaN -> 0 (sin información de afinidad)
  return(sim)
}
S_user <- similitud_usuarios(M)
```

Predicción centrada en la media

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in V} \text{Sim}(u, v) (r_{vi} - \bar{r}_v)}{\sum_{v \in V} |\text{Sim}(u, v)|}$$

Por qué centrar: usuarios “blandos” que siempre puntúan alto inflarían las predicciones. Restar la media propia neutraliza el sesgo. Sumar al final $\bar{r}_u$ devuelve la predicción a la escala personal del usuario objetivo.

```

predecir_voto <- function(M, S, usuario, item) {
  idx_u <- which(rownames(M) == usuario)
  idx_i <- which(colnames(M) == item)

  if (!is.na(M[idx_u, idx_i])) return(M[idx_u, idx_i])    # ya lo vio

  medias <- rowMeans(M, na.rm = TRUE)
  media_u <- medias[idx_u]

  otros_votos <- M[, idx_i]
  similitudes <- S[idx_u, ]
  validos <- !is.na(otros_votos) & (rownames(M) != usuario)

  if (sum(validos) == 0) return(NA)

  votos_centrados <- otros_votos[validos] - medias[validos]
  num <- sum(similitudes[validos] * votos_centrados)
  den <- sum(abs(similitudes[validos]))

  if (den == 0) return(media_u)
  prediccion <- media_u + num / den
  return(max(1, min(5, prediccion)))    # acotar al rango legal
}

```

Top-N (recomendar las 3 mejores series)

[CAE EN EXAMEN] — Ejercicio 4.2 del PP1:

```

items_no_vistos <- colnames(M)[is.na(M["Usuario_1", ])]
predicciones <- sapply(items_no_vistos,
  function(item) predecir_voto(M, S_user, "Usuario_1", item))
top3 <- sort(predicciones, decreasing = TRUE)[1:3]

```

Cold Start (te lo van a preguntar)

Usuario nuevo sin valoraciones todas las similitudes son NA/0 `predecir_voto` devuelve NA. **Soluciones:** pedirle al usuario que valore 5 ítems al registrarse, recomendarle el top de popularidad global (PageRank), o usar contenido (basado en metadatos) en lugar de colaborativo.

12. Item-Based CF (Amazon)

Misma lógica pero similitud entre **columnas** (ítems). Más estable porque el catálogo cambia menos que la base de usuarios — **patente de Amazon**.

```
S_item <- cor(M, use = "pairwise.complete.obs")
S_item[is.na(S_item)] <- 0
# Si a un usuario le gusta Matrix (cor=0.98 con Inception),
# le recomendamos Inception antes que Titanic (cor=-1).
```

13. Factorización SVD (modelos de factores latentes)

Idea: $R \approx U\Sigma V^T$. Comprimimos la matriz dispersa en k factores latentes (“Acción/Drama”, “Profundidad emocional”, ...). Es la técnica del ganador del Netflix Prize (2009).

Paso 1 — Imputación por columna

[CAE EN EXAMEN] — PP2 obliga a esto antes de SVD:

```
M_filled <- M
for (i in 1:ncol(M)) {
  M_filled[is.na(M_filled[, i]), i] <- mean(M_filled[, i], na.rm = TRUE)
}
```

Paso 2 — Descomposición y truncamiento

```
svd_res <- svd(M_filled)      # devuelve $u, $d, $v
k <- 2                        # nº de factores latentes
U_k <- svd_res$u[, 1:k]
D_k <- diag(svd_res$d[1:k])
V_k <- svd_res$v[, 1:k]
M_pred <- U_k %*% D_k %*% t(V_k)
rownames(M_pred) <- rownames(M); colnames(M_pred) <- colnames(M)

# Predicción para Usuario_1 / The Office:
M_pred["Usuario_1", "The Office"]
```

Paso 3 — Interpretación de los factores latentes

[CAE EN EXAMEN] — PP2 Ejercicio 4.2 pregunta exactamente esto:

- **Factor 1:** si todas las series cargan en la misma dirección (todos signos negativos o todos positivos) **representa popularidad / volumen de votos global**, NO un género. Es la dimensión que captura “cuánta gente ha votado este ítem”.
- **Factor 2 (y siguientes):** separa los géneros / temáticas. Las series con coordenadas más positivas vs. más negativas en el eje *Y* revelan **clusters de gusto latente** (ej.: ciencia ficción vs. comedia).
- **El “milagro”:** SVD **NO conoce el género** previamente. Lo descubre puramente de los patrones de co-rating: si los mismos usuarios votan alto a A y a B, A y B caen cerca en el espacio latente.

Visualización del espacio latente

```
library(ggrepel)
df_latente <- data.frame(
  Serie = colnames(M),
  Factor1 = svd_res$v[, 1],
  Factor2 = svd_res$v[, 2]
)
ggplot(df_latente, aes(x = Factor1, y = Factor2, label = Serie)) +
  geom_point(color = "steelblue", size = 3) +
  geom_text_repel(size = 3) +
  labs(title = "Series en el espacio latente (SVD k=2)")
```

14. Recomendación como problema de grafos

Punto pivote: “¿Qué película le gustará a Ana?” “¿Qué nodo se conectará con Ana?” = **Link Prediction**.

Grafo bipartito usuario–ítem

```
g_bip <- graph_from_data_frame(interacciones[, c("usuario","item")], directed = FALSE)
V(g_bip)$type <- V(g_bip)$name %in% interacciones$item # TRUE = ítem
is_bipartite(g_bip)
biadj <- as_biadjacency_matrix(g_bip)
```

Proyecciones unimodales

[PROYECTO AMAZON] — Tu grafo de coparticipación de productos es exactamente esto.

```
proyecciones <- bipartite_projection(g_bip)
g_usuarios <- proyecciones$proj1 # peso = nº ítems compartidos
g_items <- proyecciones$proj2

# Aplicar umbral N del enunciado
g_items_N <- delete_edges(g_items, E(g_items)[weight < N])
g_items_N <- delete_vertices(g_items_N, V(g_items_N)[degree(g_items_N) == 0])
```

PageRank Personalizado (PPR)

$$\pi = d \cdot M\pi + (1 - d) \cdot v_{\text{personalización}}$$

El vector v pone toda la “masa de teletransporte” en el usuario objetivo: el paseo aleatorio se desliza por la red, pero siempre vuelve a casa.

```
idx_ana <- which(V(g_bip)$name == "Ana")
pers_vec <- rep(0, vcount(g_bip)); pers_vec[idx_ana] <- 1

ppr_ana <- page_rank(g_bip, directed = FALSE, personalized = pers_vec)$vector
```

PPR vs PageRank global: PR global = popularidad para todos; PPR = relevancia para Ana. La diferencia entre rankings es el valor de la personalización.

Similitudes para Link Prediction

```
sim_jac <- similarity(g, method = "jaccard") # |N(i) N(j)| / |N(i) N(j)|
sim_dice <- similarity(g, method = "dice") # 2|N(i) N(j)| / (|N(i)| + |N(j)|)
sim_aa <- similarity(g, method = "invlogweighted") # Adamic-Adar
```

[PROYECTO AMAZON] — Para el reto “predecir co-compra”: calcula Jaccard / Adamic-Adar entre todos los pares no conectados y recomienda los pares con score más alto.

15. Métricas de Evaluación (Top-N)

Punto clave: en recomendación NO usamos RMSE, sino **calidad del ranking superior**.

Precision@K y Recall@K

$$\text{Precision@K} = \frac{|\text{Top-K} \cap \text{Relevantes}|}{K} \quad \text{Recall@K} = \frac{|\text{Top-K} \cap \text{Relevantes}|}{|\text{Relevantes}|}$$

```
precision_at_k <- function(reco, rel, K) sum(head(reco, K) %in% rel) / K
recall_at_k    <- function(reco, rel, K) sum(head(reco, K) %in% rel) / length(rel)
```

Trade-off: K grande Recall sube, Precision cae. Streaming → prima Recall; medicina → prima Precision.

MAP — Mean Average Precision

Premia que los aciertos estén **arriba** del ranking:

```
map_at_k <- function(reco, rel, K) {
  top_k <- head(reco, K)
  es_rel <- top_k %in% rel
  if (!any(es_rel)) return(0)
  precisions <- cumsum(es_rel) / seq_along(es_rel)
  sum(precisions * es_rel) / length(rel)
}
```

NDCG — Normalized Discounted Cumulative Gain

Penaliza posiciones bajas con descuento logarítmico. $\text{NDCG} \in [0, 1]$.

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad \text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)}$$

```
ndcg_at_k <- function(reco, rel, K) {  
  top_k <- head(reco, K)  
  gains <- ifelse(top_k %in% rel, 1, 0)  
  dcg <- sum(gains / log2(seq_along(gains) + 1))  
  ideal <- head(c(rep(1, length(rel)), rep(0, K)), K)  
  idcg <- sum(ideal / log2(seq_along(ideal) + 1))  
  if (idcg == 0) 0 else dcg / idcg  
}
```

Cobertura del catálogo

$$\text{Cobertura} = \frac{\text{ítems distintos jamás recomendados}}{|I|}$$

Baja cobertura sesgo de popularidad “long tail” abandonado.

PARTE III — Recordatorio rápido ggplot2 (suele caer integrado)

[CAE EN EXAMEN] — Patrón estándar (ejemplo txhousing / sentimientos):

```
ggplot(datos, aes(x = date, y = median, color = city)) +  
  geom_line(linewidth = 0.6) +  
  geom_point(size = 1, alpha = 0.5) +  
  scale_color_viridis_d(option = "plasma", name = "Ciudad") +  
  scale_y_continuous(labels = scales::label_dollar()) +  
  labs(title = "Evolución del Precio de la Vivienda en Texas",  
       subtitle = "Precio mediano de venta (2010-2015)",  
       x = "Año", y = "Precio Mediano ($)", caption = "Fuente: TAMU") +  
  theme_minimal(base_size = 14) +  
  theme(plot.title = element_text(face = "bold", color = "#2c3e50"),  
       legend.position = "top",  
       panel.grid.minor = element_blank())
```

Las 5 capas que se piden saber explicar:

1. `ggplot(data, aes(...))` — datos + mapeo.
 2. `aes()` — qué variable va a qué propiedad visual (x, y, color, fill, size).
 3. `geom_*()` — capas físicas (point, line, col, bar, hist, hull...).
 4. `scale_*()` — cómo se traducen los datos a colores/ejes.
 5. `theme()` — todo lo que NO está relacionado con datos (fondos, fuentes, leyendas).
-

PARTE IV — Checklist específico del PROYECTO AMAZON

Bloque II — SNA

1. Grafo de coparticipación

- Bipartito usuarios productos desde `data/datos_seleccionados_limpios.csv`.
- `bipartite_projection()` → grafo de productos.
- Justificar umbral N: barre valores, mide `edge_density()` + nº de nodos no aislados, elige el codo.

2. Centralidades obligatorias

- `centrality_pagerank()` → productos faro.
- `centrality_betweenness()` → puentes entre categorías.
- **Cruce con biplot PCA del Bloque I:** ¿los outliers PCA son los hubs del grafo?

3. Comunidades

- Louvain o Walktrap con `set.seed(2026)`.
- Sensibilidad: variar `resolution` (Louvain) o `steps` (Walktrap).
- Comparar `modularity()` y `length(comm)`.

4. Visualización

- Pre-filtrar por k-core o top-PageRank.
- `layout_with_fr` solo si subgrafo > 3000 nodos; si no, `layout_with_drl`.

Bloque III — Link-based

1. **Predicción de categoría por homofilia:** a un nodo sin categoría le asignas la categoría modal entre sus vecinos. Si no hay `category`, usa la comunidad de Louvain como categoría latente.
2. **Link Prediction (co-compra):** Jaccard o Adamic-Adar entre pares no conectados.
3. **Usuarios Elite:** top-X% por PageRank de la proyección de usuarios.

Reproducibilidad (no negociable)

- `set.seed(2026)` antes de Louvain, Walktrap, `slice_sample()`, train/test, P3 .
 - `renv::snapshot()` al añadir paquete nuevo (igraph, tidygraph, ggraph, ggforce, tidytext...).
 - Rutas relativas siempre: "data/...".
 - `quarto render end-to-end` antes de cada entrega.
-

Apéndice A — Patrones canónicos de examen (moldes para copiar)

Molde 1: Crear grafo desde dos data.frames (Airports, Reddit, Tronos)

```
# Renombrar columnas a from/to y ponerlas las dos primeras
aristas <- aristas |> rename(from = ColOrigen, to = ColDestino) |>
  select(from, to, everything())

g <- graph_from_data_frame(d = aristas, vertices = nodos, directed = TRUE)
g_tidy <- as_tbl_graph(g)
```

Molde 2: Inspección anatómica (Reto 2 Reddit)

```
vertex_attr_names(g); edge_attr_names(g)
V(g)[1];               vertex_attr(g, index = 1)
E(g)[15];              edge_attr(g, index = 15)
tail_of(g, E(g)[15])$score # score del autor origen
```

Molde 3: Topometría global (Reto 3 Reddit)

```
edge_density(g)
diameter(g)
transitivity(g, type = "global")
mean_distance(g)
count_components(g)
```

Molde 4: Bloque de centralidades (Reto 4 Reddit)

```
g_tidy <- g_tidy |>
  activate(nodes) |>
  mutate(grado      = centrality_degree(),
         betweenness = centrality_betweenness(),
         pagerank    = centrality_pagerank(),
         closeness   = centrality_closeness(),
         eigen       = centrality_eigen())

# Líder por PageRank:
g_tidy |> activate(nodes) |> as_tibble() |>
  arrange(desc(pagerank)) |> slice(1)
```

Molde 5: Filtrado por grado (Reto 2 Airports)

```
g_hubs <- induced_subgraph(g, vids = which(degree(g) > 3))
# Equivalente con resta directa:
g_hubs <- g - V(g)[degree(g) <= 3]
```

Molde 6: Estilización condicional (Reto 4 Airports)

```
V(g)$size <- V(g)$pasajeros_m / 4
V(g)$label <- V(g)$ciudad
E(g)$color <- ifelse(E(g)$distancia > 2000, "red", "grey80")
plot(g, layout = layout_with_fr(g))
```

Molde 7: Análisis de sentimiento (sentimientos vivienda)

```
tokens <- textos |> unnest_tokens(word, texto)
resultado <- tokens |>
  inner_join(lexicon_esp, by = "word") |>
  group_by(id) |> summarise(sentimiento = sum(valor))

# Por bando:
```



```
tokens |> inner_join(lexicon_esp) |>
  group_by(bando) |> summarise(sentimiento = sum(valor))
```

Molde 8: Red semántica (sentimientos vivienda)

```
word_cors <- tokens |>
  filter(nchar(word) > 3) |>
  anti_join(stop_words_es, by = "word") |>
  pairwise_cor(word, id, sort = TRUE)

red <- word_cors |> filter(correlation > 0) |>
  as_tbl_graph(directed = FALSE) |>
  mutate(comunidad = group_louvain())

ggraph(red, layout = "fr") +
  geom_mark_hull(aes(x = x, y = y, fill = comunidad, group = comunidad),
    alpha = 0.1, color = NA) +
  geom_edge_link(aes(edge_alpha = correlation), edge_colour = "gray80") +
  geom_node_point(aes(color = comunidad), size = 5) +
  geom_node_text(aes(label = name), repel = TRUE) +
  theme_graph()
```

Molde 9: Recomendador User-Based completo (PP1)

```
# Funciones:
similitud_usuarios <- function(M) {
  sim <- suppressWarnings(cor(t(M), use = "pairwise.complete.obs"))
  sim[is.na(sim)] <- 0; sim
}

predecir_voto <- function(M, S, usuario, item) {
  # ... (ver Sección 11)
}

S_user <- similitud_usuarios(M)
predecir_voto(M, S_user, "Usuario_1", "The Office")

# Top-3:
items_no_vistos <- colnames(M)[is.na(M["Usuario_1", ])]
```

```
preds <- sapply(items_no_vistos, predecir_voto,
                M = M, S = S_user, usuario = "Usuario_1")
sort(preds, decreasing = TRUE)[1:3]
```

Molde 10: SVD con imputación (PP2)

```
M_filled <- M
for (i in 1:ncol(M)) M_filled[is.na(M_filled[, i]), i] <- mean(M_filled[, i], na.rm = TRUE)

svd_res <- svd(M_filled)
k <- 2
U_k <- svd_res$u[, 1:k]; D_k <- diag(svd_res$d[1:k]); V_k <- svd_res$v[, 1:k]
M_pred <- U_k %*% D_k %*% t(V_k)
rownames(M_pred) <- rownames(M); colnames(M_pred) <- colnames(M)
M_pred["Usuario_1", "The Office"]
```

Apéndice B — Errores y warnings de igraph 2.x

Si copias código antiguo, R te avisará. Cambia siempre a la nueva sintaxis:

Antigua	Nueva
graph()	make_graph()
graph.adjacency()	graph_from_adjacency_matrix()
induced.subgraph()	induced_subgraph()
as_incidence_matrix()	as_biadjacency_matrix()
hub_score(), authority_score()	hits_scores() (devuelve lista con \$hub y \$authority)
get.edgelist()	as_edgelist()

Trampas conceptuales

- `closeness()` en grafos dirigidos premia falsamente a nodos marginales → **usar `harmonic centrality()`**.
- Louvain y Walktrap son **estocásticos**: sin `set.seed()` cada compilación da comunidades distintas.

- `bipartite_projection()` necesita `V(g)$type` definido (TRUE/FALSE), si no falla silenciosamente.
- `centrality_*`() SOLO dentro de `mutate()` / `filter()` sobre `tbl_graph`. Fuera, usar la versión clásica de `igraph`.
- `page_rank(..., personalized = v)`: el vector v debe tener longitud `vcount(g)`, no `n_usuarios`.
- `cor()` opera entre COLUMNAS $\rightarrow$ para similitud entre USUARIOS hay que **transponer** la matriz: `cor(t(M))`.
- En la fórmula a mano de Pearson: usar SOLO los ítems comunes para el cálculo (no la media global del usuario).

Apéndice C — Cheatsheet de funciones

Construcción y manipulación de grafos

```
graph_from_data_frame(d, vertices, directed)
graph_from_adjacency_matrix(A, mode)
make_graph(edges = c(...), n, directed)
make_graph(~ A-B-C-A)
make_graph("Zachary")
sample_gnp(n, p); sample_pa(n, power, m); play_islands(...)

V(g); E(g); vcount(g); ecount(g)
vertex_attr_names(g); edge_attr_names(g)
tail_of(g, e); head_of(g, e)

induced_subgraph(g, vids)
delete_vertices(g, V(g)[cond])
delete_edges(g, E(g)[cond])
simplify(g)
coreness(g)
```

Centralidades y comunidades

```

degree(g);                centrality_degree()
betweenness(g);           centrality_betweenness()
closeness(g);             centrality_closeness()
harmonic_centrality(g)
eigen_centrality(g)$vector; centrality_eigen()
page_rank(g)$vector;      centrality_pagerank()
hits_scores(g)            # $hub, $authority

cluster_louvain(g);        group_louvain()
cluster_walktrap(g);       group_walktrap()
cluster_infomap(g);        group_infomap()
cluster_edge_betweenness(g); cluster_fast_greedy(g)
membership(comm); sizes(comm); modularity(comm); length(comm)

```

Topometría global

```

edge_density(g); diameter(g); mean_distance(g)
transitivity(g, type = "global" | "local")
count_components(g); components(g)
articulation_points(g); cliques(g, min)
shortest_paths(g, from, to)$vpath

```

Bipartitos y link prediction

```

is_bipartite(g)
bipartite_projection(g)          # $proj1, $proj2
as_biadjacency_matrix(g)
similarity(g, method = "jaccard" | "dice" | "invlogweighted")

```

NLP / textos

```

unnest_tokens(data, output, input)
anti_join(stop_words_es, by = "word")
get_stopwords(language = "es")
wordStem(word, language = "spanish")
pairwise_cor(data, item, feature, sort = TRUE)

```

Recomendadores

```
similitud_usuarios(M) # Pearson entre usuarios
predecir_voto(M, S, usuario, item) # centrada en la media
M_filled <- imputar_por_columna(M) # bucle for sobre ncol
svd_res <- svd(M_filled) # $u, $d, $v
M_pred <- U_k %*% D_k %*% t(V_k) # truncamiento k
precision_at_k(); recall_at_k(); map_at_k(); ndcg_at_k()
```

Visualización con ggraph

```
ggraph(g_tidy, layout = "fr") + # o "stress", "kk", "circle", "bipartite"
  geom_edge_link(aes(edge_alpha = weight), edge_colour = "gray80") +
  geom_node_point(aes(size = grado, color = comunidad)) +
  geom_node_text(aes(label = name), repel = TRUE) +
  geom_mark_hull(aes(x = x, y = y, group = comunidad, fill = comunidad), alpha = 0.1) +
  theme_graph()
```

Fin del manual del parcial 2.